Evaluating Trust Boundaries in MCP Apps: An Experimental Study of Host-Mediated Capabilities

Research Proposal

Emirates International University

Prof. Dr. B. Malik Nasir Ali Al-Jabri; Mubarak Walid Al-Hamadi; Yasin Abd Al-Aziz Al-Khalqi; Muhammad Saif Ali Al-Heimi

2026 / 8

Abstract

MCP Apps let an MCP server deliver interactive user interfaces that a host renders inside a sandbox. The sandbox isolates the UI from the host, but the UI can still request host-mediated capabilities such as opening a link, sending a chat message, or updating the model context. Protection therefore depends not only on isolation but also on the policy decisions the host applies to these requests (Salomon et al., 2026). This proposal presents a reproducible experimental study of the boundary between what isolation technically forbids and what the host must govern by policy. We compare a specification-compliant host (baseline) with the same host augmented by a small host-side policy layer (treatment), measuring security, continued legitimate functionality, and processing latency. We state three testable hypotheses and an honest proof-of-concept scope.

1. Introduction

The Model Context Protocol (MCP) lets AI applications interact with external tools and resources in a uniform way. The MCP Apps extension adds interactive interfaces delivered by an MCP server and rendered by the host inside an isolated environment, instead of merely returning text and data (Hou et al., 2025; Model Context Protocol, 2026b).

The host sandbox protects the host from direct access to its page and data, but an MCP App interface can request capabilities that the host executes, such as opening a link, sending a message to the conversation, or updating the model context. Protection therefore does not stop at isolation; it also depends on the decisions and policies the host applies to these requests (Salomon et al., 2026).

The study's importance lies in focusing on the boundary between what isolation technically forbids and what the host must regulate by its policies. The study will present a practical test of this boundary, then compare a specification-compliant host with the same host after adding a specific policy layer, thereby helping reduce risk while preserving legitimate functionality.

2. Related Work

MCP security studies have focused on server and tool risks, permission management, and indirect prompt injection. They show that connecting models to tools can lead to unwanted execution or data disclosure if trust and permissions are not set correctly (Hou et al., 2025; Maloyan & Namiot, 2026; Radosevich & Halloran, 2025). AgentDojo and AgentBound introduced security-testing methods and execution-boundary enforcement, but did not focus on MCP App interface messages to the host (Bühler et al., 2026; Debenedetti et al., 2024).

The MCP Apps specification and SEP-1865 define isolation, CSP, tool visibility, and host-mediated channels, but they present design requirements rather than an experimental comparison of an additional hardening layer (Model Context Protocol, 2026b; Salomon et al., 2026).

2.1 Brief Comparison of Prior Work

Authors / Year	Title	Strengths	Weaknesses / Gaps
Model Context Protocol; Salomon et al. (2026)	MCP Apps Specification & SEP-1865	Define the formal architecture, threat model, and controls (CSP, tool visibility, host channels).	No experimental before/after comparison with an added host policy layer.
Maloyan & Namiot (2026)	Breaking the Protocol	Specification analysis; experiments on injection and trust.	Focuses on base MCP, not the interactive App channels.
Bühler et al. (2026)	AgentBound	Enforces policies on MCP server execution.	Addresses system level, not interface messages.
Hou et al. (2025)	MCP: Landscape,	Broad taxonomy	Does not focus on

	Security Threats, and Future Research Directions	of MCP risks and server lifecycle.	MCP Apps interfaces and host decisions.
Radosevich & Halloran (2025)	MCP Safety Audit	Practical audit of MCP server and tool risks.	Does not test the boundary between MCP App interface and host.
Debenedetti et al. (2024)	AgentDojo	Measures security while preserving legitimate work.	General agent environment, not an MCP host / Apps setting.
Greshake et al. (2023)	Not What You've Signed Up For	Shows indirect prompt-injection risk.	Not specific to MCP Apps.

2.2 Research Gap

Within the sources we reviewed, prior studies focused on MCP server and tool security, or on prompt injection and agent isolation. We found no focused experimental study that compares a specification-compliant MCP Apps host with the same host after enabling an additional policy to govern host-mediated capabilities, while measuring security, preservation of functionality, and performance. This work addresses that gap by building a reproducible test environment and a small policy-layer prototype.

3. Aim and Contribution

3.1 Research Aim

To measure the protection boundary in MCP Apps in practice, identify the difference between mandatory isolation requirements and decisions that depend on host policy, then test a preliminary solution that reduces misuse of allowed channels.

3.2 Expected Contribution

- A reproducible test suite that separates specification-compliance tests from residual-risk tests.
- A host-side policy-layer prototype that governs links, messages, and model-context updates and makes the interface's origin explicit.

- A before/after comparison in terms of security, continued legitimate functionality, and processing latency.

Novelty note: the individual controls (link allow-lists, explicit consent, origin display, rate/size limits, audit logging) are good practice rather than inventions. The novel element of this work is twofold: (a) the empirical quantification of residual risk that remains possible through channels the specification explicitly allows, and (b) the trust-boundary marker treated as a measurable UI affordance whose effect is evaluated rather than assumed.

4. Research Problems

4.1 Main Problem

MCP Apps isolate the interface from the host but allow it to request operations the host executes. The literature we reviewed lacks a simple experimental evaluation showing what the specification forbids mandatorily, what may remain possible through allowed channels, and how effective an added host policy is at reducing that risk.

4.2 Sub-Problems

- SP1: the need to verify practically that the host complies with CSP boundaries and tool/resource visibility.
- SP2: the possibility of misusing allowed channels such as ui/open-link, ui/message, and ui/update-model-context without breaking the sandbox.
- SP3: the need to know whether an added policy layer reduces unwanted behavior without disabling legitimate work or adding large latency.

4.3 Hypotheses and Proposed Solution

Hypothesis (testable)	Problem
H1: The specification-compliant host will reject all test cases violating CSP and tool/resource visibility, and will allow legitimate compliant cases. (SP1)	SP1
H2: The proposed policy layer will reduce the number of sensitive actions executed without approval and block transfer of the synthetic value through a non-sanctioned	SP2

channel, compared with baseline. (SP2)	
H3: The layer will preserve all predefined legitimate control cases, and the median added local processing overhead will not exceed 20 ms. (SP3)	SP3

Baseline behavior clarification: the specification-compliant baseline host implements the MUST-level sandbox, CSP, and tool/resource-visibility requirements, but issues no mandatory approval prompt for ui/open-link, ui/message, or ui/update-model-context. Absent the policy layer, it dispatches these requests per the specification's default handling (opens the external link, delivers the message to the chat, updates the model context). This default dispatch is precisely the residual-risk surface the policy layer targets.

Proposed solution: a layer inside the host intercepts sensitive requests before execution and applies a link allow-list, explicit consent, interface-origin display, size and rate limits, and an audit log. Hypotheses are accepted or rejected per actual results; the study does not assume the solution succeeds in advance.

5. Importance and Objectives

5.1 Importance of the Study

The study shows the host's responsibility when handling an interface delivered by an untrusted server, and prevents confusion between sandbox protection and host-policy protection. It also offers a test and a prototype that MCP host developers can reuse.

5.2 General Objective

To evaluate trust boundaries in MCP Apps, build a small host-side policy layer, and measure its effect experimentally.

5.3 Specific Objectives

- Extract the mandatory security requirements and the host-policy-dependent channels.
- Build a local lab, establish a unified baseline, and define repeatable scenarios.
- Implement the proposed policy layer and run the same scenarios after enabling it.

- Compare security, functionality, and performance, and accept or reject the hypotheses per results.

6. Methodology

6.1 Study Type and Design

The study uses an applied experimental method inside an isolated local lab. The same cases are executed in two groups:

- Baseline: a test host implementing the MUST requirements of the MCP Apps specification, without the extra policy layer.
- Treatment: the same host and environment, with only the proposed layer enabled.

The independent variable is the layer state (off/on). The dependent variables are allow/deny decisions, execution of actions without approval, access to synthetic data, success of legitimate functionality, and processing latency.

6.2 Test Environment and Scope

- A single Linux machine in an isolated local environment.
- One test host built on basic-host and AppBridge from the ext-apps project (Model Context Protocol, 2026a).
- A local MCP server, a legitimate interface, and an untrusted test interface.
- Dummy tools and resources, a local receiver server, and a test value such as SYNTHETIC_SECRET_001.
- A unified log of JSON-RPC messages, policy decisions, and timings.

The attacker is assumed to control the MCP server and interface but has no direct access to the user's device. The study does not break the browser, escape the sandbox, or run phishing on real people.

6.3 Proposed Solution

The policy layer intercepts interface messages before execution and applies four rules:

- 1. Links: protocol scheme validation (http/https) and explicit user consent prior to dispatching window.open.
- 2. Messages and context: explicit user consent, origin source badge display, and message size caps (4 KB limit).

- 3. Visible trust boundary: add a persistent marker around the interface stating it is delivered by an external MCP server.
- 4. Audit: log source, request type, decision, reason, and processing time without storing sensitive data.

CSP and tool-visibility tests remain part of baseline specification compliance, not a new contribution of the layer. The novel, evaluated element here is the measurable trust-boundary marker and the empirical residual-risk measurement through allowed channels.

6.4 Implementation Steps

- 1. Run the official example; verify a legitimate interface displays and an allowed tool is called.
- 2. Build the local server, the two interfaces, dummy tools/resources, and the receiver server.
- 3. Install test cases; run them in Baseline and record results.
- 4. Implement the policy layer as a toggleable module.
- 5. Run the same cases in Treatment with identical settings and data.
- 6. Compare results and test H1-H3; write conclusions.

6.5 Test Matrix

Test	Baseline (expected)	Treatment (expected)	Metric
Legitimate tool with app access	Allow	Allow	Function preserved
Non-visible tool to the app	Reject	Reject	Spec compliance
Connection to a domain not declared in CSP	Reject	Reject	CSP compliance
Open external link	No approval prompt; link opened per default spec handling (observed/logged)	Consent then allow/deny	Execution without approval
ui/message or update-model-context to host	Delivered with no origin indicator (observed/logged)	Consent and source shown	Channel integrity
Host-	Rendered inside	Persistent source	Trust-boundary

impersonating interface	isolation	marker shown	clarity
Predefined legitimate operation	Success	Success	False block

We study three host-mediated capability categories: (1) link opening (ui/open-link), (2) messaging and model-context update (ui/message, ui/update-model-context), and (3) audit/observability. The exfiltration test places SYNTHETIC_SECRET_001 in a ui/message body and in a ui/update-model-context payload and checks whether it reaches the local receiver.

6.6 Measurement and Analysis

- Number of successful compliance tests out of the total.
- Number of sensitive requests executed without approval before and after the layer.
- Success or failure of synthetic-value transfer per channel.
- Ratio of successful legitimate operations and count of falsely blocked operations.
- Audit-log completeness and mean/median processing time at the 95th percentile.

Hypothesis acceptance plan: H1 is accepted if Baseline rejects 100% of CSP-violating and tool-non-visible cases and allows 100% of legitimate compliant cases; otherwise rejected. H2 is accepted if Treatment reduces sensitive actions without approval to zero and blocks transfer of SYNTHETIC_SECRET_001 via ui/message and ui/update-model-context; otherwise rejected. H3 is accepted if all predefined legitimate control cases succeed in Treatment and the median local policy overhead is <= 20 ms; otherwise rejected.

Each functional case runs five times per group. Performance is measured 30 times after five warm-up runs that are excluded. Results are shown as counts, percentages, and milliseconds, then compared with the H1-H3 expectations. The 20 ms figure is an initial operational threshold set by the researchers for in-lab evaluation, not a normative limit in the MCP Apps specification.

6.7 Ethical Controls and Limitations

All experiments run locally with synthetic data and accounts, and target no public services or real users. Results are limited to one host and three capability

categories; the prototype is therefore a proof of concept, not a finished security product.

7. Time Plan

The research runs over six weeks.

Period	Main Work	Output	Direct Cost
Week 1	Review specification; run official example; fix test plan.	Working lab + fixed test plan	0
Week 2	Build the two interfaces; run Baseline cases.	Baseline results	0
Week 3	Complete Baseline measurements; finalize Baseline logs.	Finalized Baseline results	0
Week 4	Implement the policy layer as a toggleable module.	Policy-layer module	0
Week 5	Run the same cases in Treatment.	Post-solution results	0
Week 6	Analyze results; test H1-H3; write report.	Final report	0

No direct cost is expected: the machine is available, tools are open source, and experiments are local. Any contingent cost is documented in the actual report.

8. References

Bühler, C., Biagiola, M., Di Grazia, L., & Salvaneschi, G. (2026). AgentBound: Securing execution boundaries of AI agents. Proceedings of the 34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. https://doi.org/10.1145/3808103

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection

attacks and defenses in LLM agents. NeurIPS 2024 Datasets and Benchmarks Track. https://arxiv.org/abs/2406.13352

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. https://arxiv.org/abs/2302.12173

Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). Model Context Protocol (MCP): Landscape, security threats, and future research directions. arXiv:2503.23278. https://arxiv.org/abs/2503.23278

Maloyan, N., & Namiot, D. (2026). Breaking the Protocol: Security analysis of the Model Context Protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents. arXiv:2601.17549. https://arxiv.org/abs/2601.17549

Model Context Protocol. (2026a). ext-apps: Specification and SDK for the MCP Apps extension [Source code]. https://github.com/modelcontextprotocol/ext-apps

Model Context Protocol. (2026b). MCP Apps specification (revision 2026-01-26). https://github.com/modelcontextprotocol/ext-apps/blob/main/specification/2026-01-26/apps.mdx

Radosevich, B., & Halloran, J. (2025). MCP safety audit: LLMs with the Model Context Protocol allow major security exploits. arXiv:2504.03767. https://arxiv.org/abs/2504.03767

Salomon, I., Yosef, L., Chafik, O., Swannack, J., Hefner, J., Pidkuiko, A., Cooper, N., Ashley, B., & Christakis, A. (2026). SEP-1865: MCP Apps—Interactive user interfaces for MCP. Model Context Protocol. https://modelcontextprotocol.io/seps/1865-mcp-apps-interactive-user-interfaces-for-mcp